Git Basics

Introduction to Computer Science – TA Lecture W9

Lecturer: Kangbo Lyu

11th Nov. 2026

Goal of this Lecture



Understand git-based version control



Save and inspect versions on your own machine



Work with branches and resolve conflicts



Collaborate through a remote repository

Outline

1. Preparation: Installation and Version Control
2. Local Workflow: init, status, add, commit, restore, log
3. Past Versions and Differences: checkout, diff
4. Branches, Merging and Undoing Changes
5. Remote Repositories and Collaboration

Part I: Installation and Version Control



Git runs on your laptop and on a server



Why do we need version control?

Git Installation

Windows: install Git from git-scm.com, then open Git Bash or cmd.exe

macOS: run git --version in Terminal and follow the prompt
if Command Line Developer Tools are missing

Debian / Ubuntu: sudo apt update, then sudo apt install git

Checkpoint: git --version prints the installed Git version.

Git for Version Control

History: record useful milestones in a project

Recovery: return to a recorded version

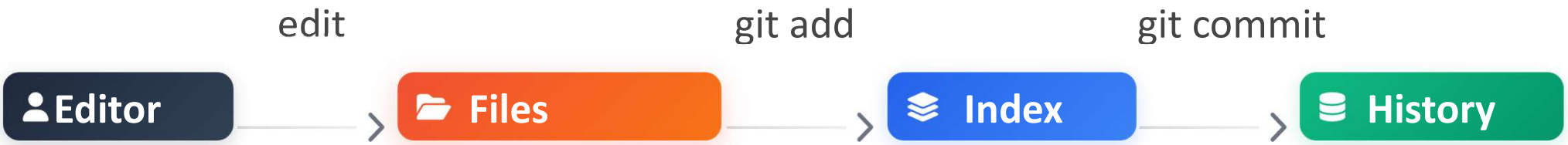
Collaboration: combine work from separate branches



Why Version Control Helps

Without Version Control	With Git
hello_final.py, hello_final_v2.py, ...	One filename with commit history
Unclear changes between saved copies	Commit messages and differences
Manual copying of teammates' files	Branches, merging and conflict resolution

Part II: The Local Git Workflow



Working tree: the project files you can open and edit

Index (staging area): the snapshot prepared for the next commit

Repository: commits and metadata stored in .git

Git Identity

Set the Name and Email Recorded in Your Commits

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Replace the example values with your own identity

Omit --global to configure only the current repository

git init: A Local Repository

```
mkdir git-practice
cd git-practice
git init -b main
```

Creates Git metadata in the .git directory

Uses main as the initial branch name

Create hello.py in your editor, then save the file

git status: What Has Changed?

```
git status
```

State	Meaning
Untracked	A file outside the index
Modified	Tracked content differs from the index
Staged	Content prepared for the next commit

One file can have both staged and unstaged changes.

git add: Prepare a Snapshot

```
git add hello.py
git status
```

Stages the file contents as they are now

Later edits need another git add to enter the next commit

git add . stages changes under the current directory

git commit: Record a Version

```
git commit -m "Add a greeting program"
git status
```

Records the prepared snapshot in the local repository

Associates it with a commit ID, author and message

Use a message that explains one meaningful change

A local commit does not upload your code to a remote repository.

git restore: Discard an Edit

For a Tracked File with Unstaged Changes

```
git restore -- hello.py
git status
```

Copies the indexed version into the working tree

Keeps the index and commit history unchanged

This overwrites unstaged changes in hello.py. Inspect them first.

git log: Inspect Commit History

```
git log
git log --oneline
```

Read the author, date and message of a commit

Use --oneline for a compact list of commit IDs and messages

Copy a commit ID when you want to inspect that version

Demonstration Video 1



Part III: Past Versions and HEAD

A commit identifies a recorded project snapshot

A branch name points to its latest commit



HEAD normally names your current branch

Checkout of a commit ID creates a detached HEAD

git checkout: Visit a Commit

Start with a Clean Working Tree

```
git status
git checkout <commit-id>
```

Inspect the historical files, then return to your branch

```
git checkout main
```

Replace <commit-id> with an ID from git log.

git diff: Which Versions Differ?

Command	Comparison
git diff	Index vs working tree
git diff --staged	HEAD vs index
git diff HEAD	HEAD vs working tree

Use -- hello.py to limit the comparison to one path.

Ordinary git diff does not display untracked file contents.

git diff: Compare Two Commits

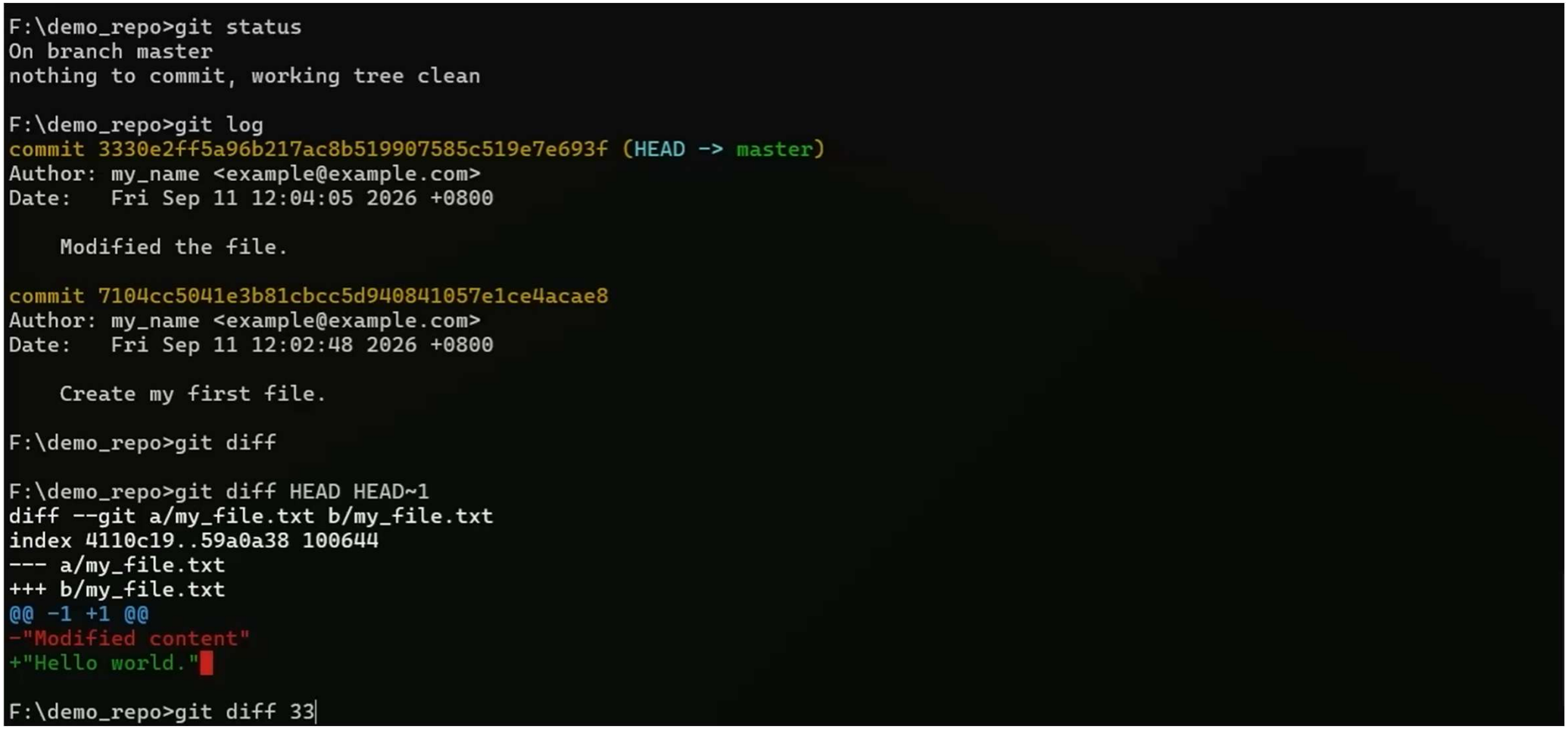
```
git diff HEAD~1 HEAD -- hello.py
```

Illustrative Diff

```
-print("Hello")
+print("Hello, Git!")
```

A minus line comes from the older version. A plus line is added.
HEAD~1 means the first parent of HEAD.

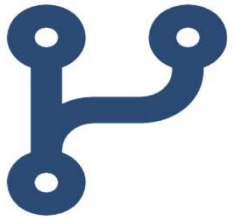
Demonstration Video 2



Part IV: Branches, Merging, and Recovery

Create a branch for one feature or experiment

New commits advance the branch you are working on



Other branch names stay at their existing commits

Merge the feature when its changes are ready

git branch and git checkout

```
git branch
git branch feature
git checkout feature
```

git branch lists branches and marks the current one with *

Creating a branch alone does not switch to it

git checkout -b feature creates and switches in one step

git merge: Combine Branches

```
git checkout main
git merge feature
```

The current branch receives the other branch’s changes

A fast-forward moves the branch tip to a later commit

Diverged histories may need a merge commit

Merging does not delete the feature branch.

When a Merge Has Conflicts

Git Needs Help When It Cannot Combine the Edits

```
<<<<<<< HEAD
print("Hello, class!")
=======
print("Hello, Git!")
>>>>>>> feature
```

In this merge, HEAD is main and feature is the incoming branch.
Both versions are available for you to review.

Resolving a Merge Conflict

Read both versions and decide the intended final behavior

Edit hello.py, remove the conflict markers, and test it

```
git add hello.py
git status
git commit -m "Merge feature and resolve greeting"
```

To abandon an unfinished merge: git merge --abort

git checkout: Restore One File

From the Index

```
git checkout -- hello.py
```

From a Specific Commit

```
git checkout <commit-id> -- hello.py
```

The first form changes the working tree.
The second changes both the working tree and index.

git add -p: Stage Part of a File

```
git add -p hello.py
```

Response	Action
y	Stage this hunk
n	Leave this hunk unstaged
s	Split the hunk when possible
q	Stop reviewing the remaining hunks

For an edited tracked file, review git diff --staged before committing.

git restore: Choose the Destination

Unstage a File and Keep Its Working Copy

```
git restore --staged -- hello.py
```

Copy an Earlier Version into the Working Tree

```
git restore --source=HEAD~1 -- hello.py
```

--staged uses HEAD by default. --source sets an explicit source.
The second command overwrites the working copy.

git reset: Revise a Local Commit

For the Latest Unpublished Commit

```
git reset --soft HEAD~1
git status
```

Moves the current branch back to its parent commit

Keeps the index and working tree as they are

Review the staged changes, then create a corrected commit

Three Modes of git reset

When Resetting the Current Branch to a Commit

Mode	Branch Tip	Index in .git	Working Files
--soft	Moves	Unchanged	Unchanged
--mixed	Moves	Matches target	Unchanged
--hard	Moves	Matches target	Matches target

--mixed is the default. --hard discards tracked uncommitted work.

Recovery Has Limits

Check status and differences before undoing work

git reflog records recent HEAD and branch movements

Uncommitted edits may be unrecoverable

Keep backups of files and preserve the .git directory



Demonstration Video 3

```
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 7104cc5 Create my first file.

F:\demo_repo>git status
HEAD detached at 7104cc5
nothing to commit, working tree clean

F:\demo_repo>cat my_file.txt
"Hello world."

F:\demo_repo>git checkout master
Previous HEAD position was 7104cc5 Create my first file.
Switched to branch 'master'

F:\demo_repo>cat my_file.txt
"Modified content"

F:\demo_repo>
```

Part V: Remote Repositories

A remote is another repository reachable through a URL

GitHub, GitLab and Gitee can host shared repositories



origin is a conventional name for a remote

Use git push to send local commits to a remote

git clone: Join an Existing Project

```
git clone <remote-url> git-practice
cd git-practice
git remote -v
```

Replace <remote-url> with the project’s clone URL

Clone creates a repository and checks out an initial branch

It normally records the source remote as origin

A First Push from Your Repository

First, Create an Empty Remote Repository

```
git remote add origin <remote-url>
git remote -v
git push -u origin main
```

Replace <remote-url> with the URL of that repository

-u sets the upstream for main, so later pushes can be shorter

git fetch: Inspect Remote Updates

```
git fetch origin
git log --oneline main..origin/main
git diff main origin/main
```

Fetch updates remote-tracking references such as origin/main

It leaves your current branch and working files unchanged

Review incoming commits before integrating them

git pull: Fetch and Integrate

```
git checkout main
git pull --ff-only
```

Updates from the configured upstream when a fast-forward works

Stops if the local and upstream histories have diverged

After inspection, git pull --no-rebase requests a merge

A Shared Branch Workflow

```
git checkout -b feature
# Edit files, then add and commit
git push -u origin feature
```

Open a pull request or merge request on the hosting platform

Review the changes and resolve conflicts before merging

Update main after the shared change is merged

Two Habits to Keep

Before a Commit

Check status and the staged diff.
Commit one meaningful change with a clear message.

Before Undoing or Sharing

Know whether the command changes files, the index or a branch.
Inspect remote updates before combining histories.